

第一章：数据分析入门

数据处理是数据分析的核心部分，通过爬虫或者实际生产过程中初步获取的数据通常具有很多的“垃圾数据”，比如重复数据或者值缺失，不连续数据等等。这时就需要对数据首先进行筛选，补全等“清洗”操作。除此之外，“清洗”好的数据也需要根据不同的用途来进行转换，以适应分析，预测或者可视化的需求。

数据的处理的软件包有很多，在python中主要应用Pandas来进行处理。Pandas是一个十分成熟的数据处理包，熟练掌握可以高效并且方便地将数据进行转换和清洗，本节主要整理了pandas的一些基本技能和实用技巧，为励志成为数据分析师的你铺路搭桥。

引言：

Pandas 是基于 NumPy 的一个开源 Python 库，它被广泛用于数据分析，以及数据清洗和准备等工作。数据科学家经常和表格形式的数据（比如.csv、.tsv、.xlsx）打交道。Pandas可以使用类似SQL的方式非常方便地加载、处理、分析这些表格形式的数据。搭配Matplotlib和Seaborn效果更好。

pandas可以满足以下需求：

- 具备按轴自动或显式数据对齐功能的数据结构。这可以防止许多由于数据未对齐以及来自不同数据源（索引方式不同）的数据而导致的常见错误、集成时间序列功能、既能处理时间序列数据也能处理非时间序列数据的数据结构、数学运算和简约（比如对某个轴求和）可以根据不同的元数据（轴编号）执行、
- 灵活处理缺失数据、
- 在实际构建任何模型之前，任何机器学习项目中的大量时间都必须花费在准备数据、
- 分析基本趋势和模式上。因此需要Pandas来进行处理。

下面我们开始今天的学习之旅。

1. Pandas入门

1.1 Pandas的安装与导入

首先，在使用Pandas前，必须安装Pandas。如果你安装过Anaconda，就可以执行如下命令安装Pandas：

```
    conda install pandas
```

如果没有安装Anaconda，也没有关系，可以使用Python的pip命令来安装：

```
    pip install pandas
```

注意：pandas安装会用到numpy库，因此在安装pandas之前一定要安装好numpy。
导入：为了简便，这里使用pd作为pandas的缩写，因为pandas依赖numpy，所以在使用之前需要安装和导入numpy

```
import numpy as np
import pandas as pd
```

打印pandas的版本

```
pd.__version__
```

考虑如下的Python字典数据和Python列表标签：

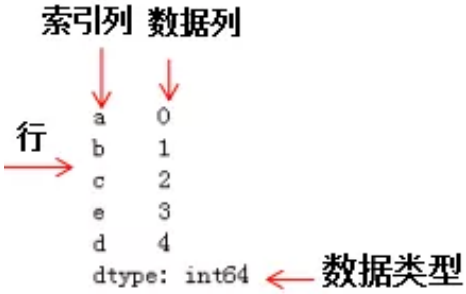
```
data = {'animal': ['cat', 'cat', 'snake', 'dog', 'dog', 'cat', 'snake', 'cat', 'dog', 'dog'],
        'age': [2.5, 3, 0.5, np.nan, 5, 2, 4.5, np.nan, 7, 3],
        'visits': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
        'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no', 'no']}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

1.2 pandas数据结构介绍

Pandas有两个数据结构：Series和DataFrame。

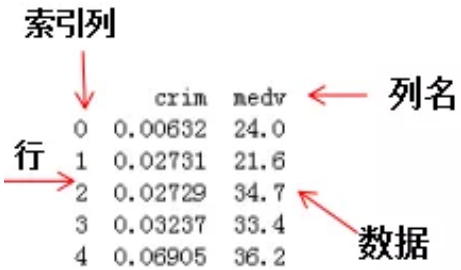
> pd.DataFrame:创建pandas矩阵

- Series是一种类似于以为NumPy数组的对象，它由一组数据（各种NumPy数据类型）和与之相关的一组数据标签（即索引）组成的。可以用index和values分别规定索引和值。如果不规定索引，会自动创建 0 到 N-1 索引。



> pd.Series:创建pandas列表

- DataFrame是一种表格型结构，含有一组有序的列，每一列可以是不同的数据类型。既有行索引，又有列索引。



1.从具有索引标签的字典数据创建一个DataFrame df.

```
df = pd.DataFrame(data,index = labels)
```

返回DataFrame的前三行

```
df.iloc[:3]
df.head(3)
```

运行结果如下：

	animal	age	visits	priority
a	cat	2.5	1	yes
b	cat	3.0	3	yes
c	snake	0.5	2	no

2.从numpy 数组构造DataFrame

```
df2 = pd.DataFrame(np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]),
                   columns=['a', 'b', 'c'])
df2
```

运行结果如下

	a	b	c
0	1	2	3
1	4	5	6
2	7	8	9

3.通过其他DataFrame来创建DataFrame df3

```
df3 = df2[["a","b","c"]].copy()
df3
```

运行结果如下：

	a	b	c
0	1	2	3
1	4	5	6
2	7	8	9

4.从csv文件中每隔n行来创建Dataframe

```
df = pd.read_csv('https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv', chunksize=50)
df2 = pd.DataFrame()

```

	age	b	chas	crim	dis	indus	lstat	medv	nox	ptratio
0	65.2	396.90	0.0	0.00632	4.0900	2.31	4.98	24.0	0.538	15.3
50	45.7	395.56	0.0	0.08873	6.8147	5.64	13.45	19.7	0.439	16.8
100	79.9	394.76	0.0	0.14866	2.7778	8.56	9.42	27.5	0.520	20.9
150	97.3	372.80	0.0	1.65660	1.6180	19.58	14.10	21.5	0.871	14.7
200	13.9	384.30	0.0	0.01778	7.6534	1.47	4.45	32.9	0.403	17.0

5.用Series创建DataFrame

```
s_1 = pd.Series(data['animal'])
s_2 = pd.Series(data['age'])
s_3 = pd.Series(data['visits'])
s_4 = pd.Series(data['priority'])
pd_2 = pd.DataFrame([s_1,s_2,s_3,s_4])
pd_2
```

运行结果如下：

	0	1	2	3	4	5	6	7	8	9
0	cat	cat	snake	dog	dog	cat	snake	cat	dog	dog
1	2.5	3	0.5	NaN	5	2	4.5	NaN	7	3
2	1	3	2	3	2	3	1	1	2	1
3	yes	yes	no	yes	no	no	no	yes	no	no

pandas处理NaN值

> dropna(axis=, how=)：丢弃NaN数据，{axis：0(按行丢弃)，1(按列丢弃)} {how：'any'(只要含有NaN数据就丢弃)，'all'(所有数据都为NaN时丢弃)}

> fillna(value=)：将NaN值都设置为value的值

> isnull()：对每各元素进行判断是否是NaN，返回结果矩阵

> np.any(matrix) == value：判断matrix矩阵中是否有value值

> np.all(matrix) == value：判断matrix矩阵中是否所有元素都是value值

pandas读取数据、导出数据

根据数据的格式，pandas提供了多种数据读取和导出的方法，如：

> 读取数据：**read_csv**、**read_table**、**read_fwf**、**read_clipboard**、**read_excel**、**read_hdf**

> 导出数据：to_csv、to_table、to_fwf、to_clipboard、to_excel、to_hdf

```
df = pd.read_csv('Q1.csv')
print(df)
df.to_csv('Q1_pandas.csv')
```

pandas合并数据

concat方法是拼接函数，有行拼接和列拼接，默认是行拼接，拼接方法默认是外拼接(并集)，拼接对象是pandas数据类型。

> 第一个参数：需要合并的矩阵

> axis：合并维度，0：按行合并，1：按列合并

> join：处理非公有 列/行 的方式，inner：去除非公有的 列/行，outer：对非公有的 列/行 进行NaN值填充然后合并

> ignore_index：是否重排行索引

```
df1 = pd.DataFrame(np.arange(12).reshape(3, 4), columns=['A', 'B', 'C', 'D'], index=[0, 1, 2])
df2 = pd.DataFrame(np.ones((3, 4)), columns=['B', 'C', 'D', 'E'], index=[1, 2, 3])
print(pd.concat([df1, df2], join='outer', ignore_index=True)) # join = {'outer', 'inner'}
print(pd.concat([df1, df2], axis=1, join_axes=[df1.index]))
print(df1.append([df2], ignore_index=True))
```

append方法在index方向连接两个DataFrame或者对DataFrame进行扩展

append 方法可以直接用list对DataFrame进行扩展。

```
df = pd.DataFrame([[1, 2], [3, 4]])
df = df.append([[1,2]])
print(df)
```

运行结果：

	0	1
0	1	2
1	3	4
0	1	2

或者也可以将两个DataFrame连接起来。

```
df = pd.DataFrame([[1, 2], [3, 4]], columns=list('AB'))
df2 = pd.DataFrame([[5, 6], [7, 8]], columns=list('AB'))
df = df.append(df2)
print(df)
```

运行结果：

	A	B
0	1	2
1	3	4
0	5	6
1	7	8

需要注意的是，append方法并不能像list的append方法一样对原来的df继续修改，而是建立了一个新的对象。如果要修改df，那么需要重新对df赋值，所以append的方法执行效率并不是很高。

Join方法 是基于Index连接DataFrame，连接方法有内连接、外连接**(左连接和右连接)****

```
caller = pd.DataFrame({'key':['A0','A1','A2','A3','A4','A5'],'B':
['B0','B1','B2','B3','B4','B5']})
other = pd.DataFrame({'key':['A0','A1','A2'],'C':['C0','C1','C2']})
caller.join(other,lsuffix='_caller',rsuffix='_other',how='inner')
```

运行结果如下：

	B	key_caller	C	key_other
0	B0	A0	C0	A0
1	B1	A1	C1	A1
2	B2	A2	C2	A2

另外，还有一种merge方法与Join方法类似，不过语法略有不同。

通过on连接两个数据集的相同列，how表示连接的方式也有内连接、外连接(左连接和右连接)

使用merge方式要求合并的两个DataFrame需要有两数据集有一个相同列（不要求数值完全相同），继续以上面数据为例，对比下区别

```
df = pd.merge(caller,other,on = ['key'],how = 'inner')
```

运行结果如下：

	key	B	C
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

参考文献：无

作者：佘亮

责编：佘亮

审稿责编：书生，周岩